

二、系統功能與介面

本專案實作一款圖形化 C++ 踩地雷遊戲，全面採用視窗化視圖與玩家互動，系統功能包含：

1. 遊戲機制與快捷鍵操作
 - 提供三種地圖難度切換，並自動調整視窗大小：
 - **[1]** 鍵(初級): 10x10 網格, 10 顆地雷。
 - **[2]** 鍵(中級): 16x16 網格, 40 顆地雷。
 - **[3]** 鍵(高級): 16x30 網格, 99 顆地雷。
 - **[R]** 鍵: 重置地圖並重新開局。
 - 開局首點安全機制: 玩家第一次點擊格子時，系統動態調整地雷位置，確保第一點不會踩雷。
 - 雙鍵(和弦展開)功能: 當某格已翻開且數字等於周圍已插旗數時，左鍵點擊該數字可翻開周圍剩下未插旗的格子。
2. 視窗內即時打字系統
 - 遊戲判定勝利時，畫面顯示「YOU WIN!」與「Enter Name:」輸入圖層。
 - 採用 `sf::Event::TextEntered` 事件監聽捕捉鍵盤輸入，替代阻斷式 `std::cin` 指令。
 - 玩家直接在遊戲視窗內打字，支援 Backspace 鍵刪除文字，打字期間視窗不凍結，維持畫面更新率，且視窗可隨時點擊關閉。
3. 地圖分級排行榜系統
 - 玩家可隨時點擊 **[L]** 鍵 切換排行榜畫面。
 - 排行榜依據地圖尺寸進行過濾與分類(如 10x10、16x30 獨立排序)，避免跨難度混合。
 - 畫面採用固定 X 座標定位對齊名次、名稱、時間、地圖大小與通關日期，前三名以不同顏色區隔。
4. AI 解雷提示功能
 - 玩家按下 **[H]** 鍵 時，程式啟動局部約束消去演算法進行地圖分析，並在對應格子上疊加半透明色塊提示：
 - 綠色色塊: 標示出根據邏輯推導 100% 安全可翻開的格子。
 - 紅色色塊: 標示出根據邏輯推導 100% 絕對是地雷的格子。
 - 玩家在雷區進行任何滑鼠點擊動作，提示色塊會自動隱藏。
5. 音效與背景音樂整合
 - 遊戲內置背景音樂(BGM)，並整合操作音效：
 - 左鍵翻開格子: 播放點擊音效。
 - 右鍵插拔旗幟: 播放插旗音效。
 - 踩到地雷失敗: 停止播放 BGM 並播放爆炸音效。
 - 順利通關獲勝: 停止播放 BGM 並播放勝利音樂。

三、開發工具與環境

- 程式語言: C++ (使用 C++11 標準, 包含 lambda 匿名函式)。
- 整合開發環境 (IDE): Microsoft Visual Studio 2022 (配置於 x64 平台與 Debug 組態模式)。
- 作業系統: Windows 10 / Windows 11。
- 外部函式庫:
 1. **SFML (Simple and Fast Multimedia Library) v2.6.1**: 用於底層雙緩衝渲染、視窗事件管理及音訊播放控制。

2. **SQLite3 Amalgamation**: 將原始碼直接納入專案編譯，於本地端建立並讀寫資料庫檔案 (minesweeper_scores.db)。
- **AI 工具**: Gemini 大語言模型 (輔助代碼架構設計、非阻塞事件循環重構、AI 提示演算法編寫與編譯連結錯誤排查)。

四、系統實作

(一) 程式架構說明 (MVC + DB Pattern)

專案將遊戲邏輯、視覺畫面與玩家控制進行解耦，主要結構分為四個模組：

1. **Model (模型層) — GameBoard 類別與 GridCell 結構體**
 - 儲存地圖二維陣列狀態，記錄每格的屬性 (是否為雷、周圍雷數、是否翻開、是否插旗)，實作布雷、周圍雷數計算、DFS 遞迴開格與勝負判定。
2. **View (視覺層) — GameView 類別**
 - 負責所有的畫面繪製，包含地圖網格、UI 文字、打字輸入顯示、分級排行榜表格繪製，以及 AI 提示矩形色塊的疊加。
3. **Controller (控制層) — GameEngine 類別**
 - 內含遊戲主迴圈 (Game Loop)，週期性呼叫事件處理、數據更新與渲染函式，負責滑鼠事件處理、座標轉換與音效狀態控制。
4. **Database Engine (資料庫管理層) — ScoreManager 類別**
 - 負責與本地端 SQLite 資料庫連線，使用預編譯陳述式 (Prepared Statements) 處理歷史紀錄的寫入與條件篩選查詢。

(二) 程式碼重點說明

1. 非阻塞視窗事件打字實作 (GameEngine.cpp)

利用 SFML 的 TextEntered 事件擷取玩家敲擊的 Unicode 編碼，避免阻塞主執行緒：

```
C++
if (isEnteringName && e.type == sf::Event::TextEntered) {
    if (e.text.unicode == 13) { // Enter 鍵
        if (playerNameInput.empty()) playerNameInput = "Anonymous"
        ScoreManager scoreDb;
        scoreDb.insertScore(playerNameInput, elapsedSeconds, board.getRows()
board.getCols());
        isEnteringName = false
        scoreSaved = true
        showLeaderboard = true
    }
}
```

```

else if (e.text.unicode == 8) { // Backspace 鍵
    if (!playerNameInput.empty()) playerNameInput.pop_back();
}
else if (e.text.unicode < 128 && e.text.unicode >= 32 && playerNameInput.size() < 12) {
    playerNameInput += static_cast<char>(e.text.unicode);
}
}
}

```

2. 排行榜分級過濾查詢 (ScoreManager.cpp)

在資料庫查詢時使用 WHERE map_size = ? 進行過濾，實作不同難度獨立排序：

```

C++
std::string sql = "SELECT id, name, clear_time, clear_date, map_size FROM leaderboards "
    "WHERE map_size = ? ORDER BY clear_time ASC LIMIT ?;";
if (sqlite3_prepare_v2(db, sql.c_str(), -1, &stmt, nullptr) == SQLITE_OK) {
    sqlite3_bind_text(stmt, 1, mapSize.c_str(), -1, SQLITE_TRANSIENT);
    sqlite3_bind_int(stmt, 2, limit);
    while (sqlite3_step(stmt) == SQLITE_ROW) {
        // 解包數據並推入 std::vector 中
    }
}
sqlite3_finalize(stmt);

```

3. AI 局部約束消去提示演算法 (GameEngine.cpp)

演算法掃描全圖，針對已翻開的數字格，根據其周圍未翻開格數與插旗數進行邏輯推導：

- 規則一：局部滿約束
當某數字格周圍的「未翻開鄰居總數」等於「該格數字本身」時，代表周圍盲區皆為地雷。將未插旗鄰居標記為 hintType = 2 (紅色地雷提示)。
- 規則二：約束釋放
當某數字格周圍的「已插旗總數」等於「該格數字本身」時，代表約束已滿足，其餘未翻開且未插旗的鄰居皆安全。標記為 hintType = 1 (綠色安全提示)。

C++

```
if (unopenedCount == cell.neighborMines && !unflaggedUnopenedNeighbors.empty()) {
```

```
    currentHint = { unflaggedUnopenedNeighbors[0].first,
```

```
unflaggedUnopenedNeighbors[0].second, 2 };
```

```
    return
```

```
}
```

```
if (flaggedCount == cell.neighborMines && !unflaggedUnopenedNeighbors.empty()) {
```

```
    currentHint = { unflaggedUnopenedNeighbors[0].first,
```

```
unflaggedUnopenedNeighbors[0].second, 1 };
```

```
    return
```

```
}
```